Studio Nx

P1-03 백엔드 공통 — 함수 설명 합본 (01~13)

2026-10-07

목차

1. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/01_errors_app_error_handler.md
2. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/02_errors_validation_error_handler.md
3. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/03_errors_http_error_handler.md
4. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/04_errors_unexpected_error_handler.md
5. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/05_security_decode_access_token.md
6. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/06_deps_get_current_user.md
7. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/07_deps_get_current_user_optional.md
8. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/08_models_user.md
9. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/09_models_inquiry.md
10. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/10_models_project.md
11. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/11_models_project_member.md
12. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/12_models_track.md
13. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/13_models_relationships.md

Backend(FastAPI/Python) 01 — `error_json()` · `app_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식
작성	Claude (AI) · 2026-10-06 · 사용자 검토
다음	02 validation error handler() (입력값 형식 오류)

1. 하는 일

서버 코드가 `raise AppError(...)` 로 던진 오류를 정해진 모양의 JSON 응답으로 바꾼다.

```
raise AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
```

→ 상태 코드 **409**, 본문:

```
{"error": {"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}}
```

함수	역할
`error_json()`	오류 응답 하나를 만든다. 앞으로 만들 처리기 02 ~ 04 도 모두 이 함수를 쓴다
`app_error_handler()`	`AppError` 가 생기면 FastAPI 가 부르는 함수. `error_json()` 으로 응답을 만든다

2. 언제 쓰나

서버가 "요청은 받았지만 처리할 수 없다"고 알릴 때마다 쓰인다. 라우터 · 서비스 코드는 `raise AppError(...)` 한 줄만 쓰고, 응답 모양은 이 함수가 만든다.

상황 (구현 작업)	던지는 오류	화면에서 일어나는 일
가입할 때 이미 쓰는 이메일 (P1-04)	`EMAIL_TAKEN` · 409	가입 화면의 이메일 칸 아래에 "This email is already in use."
로그인 없이 마이페이지 API 호출 (P1-03 `get_current_user`)	`UNAUTHORIZED` · 401	프론트 `apiFetch` 가 토큰을 재발급한 뒤 다시 보냄 (JS 02)
이메일 · 비밀번호가 틀림 (P1-05)	`INVALID_CREDENTIALS` · 401	로그인 모달에 오류 문구
다른 사람의 프로젝트를 열려고 함 (P1-13)	`FORBIDDEN` · 403	접근 거부 문구

FastAPI 가 오류를 응답으로 바꾸는 방법

순서	일어나는 일
① 서버 시작	`main.py` 가 `register_exception_handlers(app)` 를 부르고, 이 함수가 "`AppError` 는 `app_error_handler` 가 처리한다"고 등록한다
② 요청 처리 중	라우터 · 서비스 어디에서든 `raise AppError(...)`

순서	일어나는 일
③ FastAPI 가 오류를 잡음	오류의 종류(`AppError`)로 등록된 처리기를 찾는다
④ 처리기 실행	`app_error_handler(request, exc)` → JSON 응답
⑤ 브라우저	`api.js` 가 `error.code` 를 읽어 `ApiError` 로 만든다 (JS 02)

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다 (02 ~ 04 에서 쓰는 것까지 모두 있음). 이 함수들은 **`FastAPI`** · **`Request`** · **`JSONResponse`** · **`ErrorBody`** · **`ErrorResponse`** 를 쓴다. `AppError` · `ErrorCode` 는 같은 파일 안에 있다.

import	가져오는 것	쓰는 곳
`StrEnum` (파이썬 기본)	값이 글자인 열거형	`ErrorCode`
`FastAPI` · `Request`	앱 · 요청 타입	`register_exception_handlers(app)` · 처리기의 `request`
`RequestValidationError`	입력 형식 오류의 종류	02
`JSONResponse`	JSON 응답	`error_json()`
`StarletteHTTPException`	404 · 405 같은 기본 오류의 종류 (Starlette 의 `HTTPException`)	03
`ErrorBody` · `ErrorResponse`	오류 응답 모양 (Pydantic, `app/schemas/common.py`)	`error_json()`

함수

```
def error_json(
    status_code: int,
    code: str,
    message: str,
    details: dict | None = None,
) -> JSONResponse:
    body = ErrorResponse(error=ErrorBody(code=code, message=message, details=details))
    return JSONResponse(status_code=status_code, content=body.model_dump())


async def app_error_handler(request: Request, exc: AppError) -> JSONResponse:
    return error_json(exc.status_code, exc.code, exc.message, exc.details)
```

```
def register_exception_handlers(app: FastAPI) -> None:
    app.add_exception_handler(AppError, app_error_handler)
    # TODO: RequestValidationError · StarletteHTTPException · Exception 처리기 등록
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)
`app.…`	우리 프로젝트(`studionx/backend/app/`)의 파일. `app.core.errors` = `app/core/errors.py`

식	뜻
`status_code: int`	**타입 힌트**: 이 값은 정수라는 표시. 실행에는 영향이 없고 편집기 · 검사 도구가 쓴다
`dict | None = None`	사전(dict) 또는 `None`. 넘기지 않으면 `None`
`-> JSONResponse`	이 함수가 돌려주는 값의 종류
`ErrorResponse(error=ErrorBody(...))`	`schemas/common.py` 의 Pydantic 모델. 정해 둔 칸(`code` · `message` · `details`)만 받는다
`body.model_dump()`	Pydantic 모델 → 파이썬 사전. `JSONResponse` 가 이 사전을 JSON 글자로 바꾼다
`async def`	비동기 함수. FastAPI 의 오류 처리기는 이 형태로 만든다
`request: Request`	오류가 난 요청. 지금은 쓰지 않지만 FastAPI 가 항상 넘겨준다 (P1-22 에서 API · 페이지 요청을 나눌 때 씀)
`app.add_exception_handler(AppError, app_error_handler)`	"`AppError` 가 생기면 이 함수를 불러라"라고 등록

`raise AppError(...)`	`status_code`	본문 `error`
`(EMAIL_TAKEN, 409, "This email is already in use.")`	409	`{"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}`
`(VALIDATION_ERROR, 422, "Check the password requirements.", details={"password": ["upper", "special"]})`	422	`details` 에 `{"password": ["upper", "special"]}`

4. 흐름

1. 서버 시작 때 `AppError` → `app_error_handler` 를 등록한다.
2. 요청 중 `AppError` 가 생기면 FastAPI 가 `app_error_handler` 를 부른다.
3. `error_json()` 이 `{"error": {...}}` 응답을 오류에 적힌 상태 코드로 만든다.

5. 왜 이렇게

선택	이유
응답을 만드는 함수(`error_json`)를 따로 둠	처리기가 4개(이 함수 + 02 ~ 04)가 된다. 모양을 한 곳에서만 만들어야 어긋나지 않는다

선택	이유
Pydantic 모델(`ErrorResponse`)을 거쳐 만듦	칸 이름을 잘못 쓰면 바로 오류가 난다. API 문서(`/docs`)에 나오는 모양과 실제 응답이 같다
처리기는 `AppError` 의 값을 그대로 씀	상태 코드 · 문구는 오류를 던지는 곳이 가장 잘 안다. 처리기는 모양만 맞춘다
`@app.exception_handler` 대신 `add_exception_handler`	처리기를 보통 함수로 두면 시험 · 문서에서 이름으로 가리킬 수 있다. 동작은 같다
`message` 는 영어	화면이 영어다. 화면 번역은 프론트가 `code` 로 한다

6. 시험 (`tests/test_errors.py`)

시험용 작은 FastAPI 앱에 처리기만 등록하고, 일부러 `AppError` 를 던지는 주소 3개로 확인했다.

경우	결과
`EMAIL_TAKEN` · 409	상태 409, `application/json`, 본문이 정확히 `{"error": {code, message, details: null}}`
`details` 가 있는 오류 (422)	`details` 가 그대로 전달됨
`UNAUTHORIZED` · 401	`code` 가 `"UNAUTHORIZED"` — 프론트가 재발급을 시도하는 조건과 같음

전체 시험 17개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 02 — `validation_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	01 `error_json()` · `app_error_handler()` · 03 `http_error_handler()` (404 등 기본 오류)

1. 하는 일

요청의 입력값이 API 가 정한 형식과 맞지 않을 때 FastAPI 가 내는 오류를, 칸별 문구가 담긴 422 응답으로 바꾼다.

이 함수는 유효성 검사를 하지 않는다. 검사는 이미 끝났고, 이 함수는 검사에서 떨어진 결과를 정해진 응답 모양으로 바꾼다. 더 정확한 이름은 "형식 검사 실패 응답 변환기"다.

단계	하는 곳	하는 일
① 규칙 정하기	스키마 `schemas/auth.py` 의 `SignupRequest`	"email 은 글자, 255자 이하", "nickname 은 꼭 있어야 함" 같은 규칙을 적어 둔다
② 검사하기	FastAPI · Pydantic (자동)	요청이 들어오면 ① 의 규칙과 비교한다. 틀리면 `RequestValidationError` 를 만들고 라우터를 실행하지 않는다
③ 응답 만들기	**이 함수**	② 의 오류 목록을 `{"error": {"code": "VALIDATION_ERROR", "details": {칸: 문구}}}` 로 바꾼다

예: "비밀번호 8자 이상" 규칙은 이 함수가 아니라 스키마의 `Field(min_length=8)` 에 있다. 이 함수를 지워도 검사는 그대로 일어나고, 응답만 FastAPI 기본 모양(`{"detail": [...]}`)이 된다.

유효성 검사는 세 곳에서 한다

검사	하는 곳	예	오류를 내는 방법
형식 검사	스키마 (FastAPI 가 자동으로)	칸 빠짐 · 타입 · 길이 틀림	`RequestValidationError` → **이 함수**가 422 로
규칙 검사	서비스 코드 (P1-04)	비밀번호에 대문자 · 기호가 있는지 (`check_password_rules`)	`raise AppError(VALIDATION_ERROR, 422, …)` → 01 처리기
DB 확인	서비스 코드 (P1-04)	이미 쓰는 이메일 · 닉네임	`raise AppError(EMAIL_TAKEN, 409, …)` → 01 처리기

화면(`join.js`, P1-07)도 입력하는 동안 같은 규칙을 검사하지만 사용자 편의용이다. 화면 검사는 우회할 수 있으므로 실제로 막는 것은 서버 검사다.

응답 예

가입 요청에 `nickname` 이 빠지고 비밀번호가 짧으면:

```
422  {"error": {"code": "VALIDATION_ERROR",
                "message": "Check the highlighted fields.",
                "details": {"password": "String should have at least 8 characters",
                            "nickname": "Field required"}}}
```

2. 언제 쓰나

라우터 코드가 실행되기 **전에** 생기는 오류다. 라우터는 이 오류를 직접 던지지 않는다.

순서	일어나는 일
① 요청이 들어옴	예: `POST /api/v1/auth/signup` 본문 `{"email": "a@b.com", "password": "Pw!1"}`
② FastAPI 가 입력을 스키마와 비교	라우터가 받기로 한 `SignupRequest` (`schemas/auth.py`) — 칸 이름 · 타입 · 길이
③ 맞지 않으면	`RequestValidationError` 를 만들고 **라우터를 실행하지 않음**
④ 이 처리기 실행	422 + 칸별 문구
⑤ 화면	`ApiError.details` 로 받음 — 어느 칸이 틀렸는지 알 수 있음

상황	`details`
필요한 칸이 빠짐	`{"nickname": "Field required"}`
길이 · 형식이 틀림	`{"password": "String should have at least 8 characters"}`
주소의 값이 숫자가 아님 (`/projects?page=abc`)	`{"page": "Input should be a valid integer, …"}`
본문 자체가 없음	`{"body": "Field required"}`

화면 문구는 프론트가 정한다. 가입 화면은 입력할 때 이미 규칙을 검사하고 자기 문구를 보여 준다(P1-07). 이 422 는 화면 검사를 거치지 않고 API 를 직접 부른 경우를 막는 마지막 단계이고, `details` 의 영어 문구는 개발 · 디버깅용이다.

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다. 이 함수는 **`Request`** · **`RequestValidationError`** · **`JSONResponse`** 를 쓴다. `error_json` · `ErrorCode` 는 같은 파일 안에 있다.

함수

```
async def validation_error_handler(request: Request, exc: RequestValidationError) -> JSONResponse:
    details: dict[str, str] = {}
    for err in exc.errors():
        loc = err.get("loc") or ()
```

```
        field = str(loc[-1]) if loc else "body"
        details.setdefault(field, err.get("msg", "Invalid value."))
    return error_json(422, ErrorCode.VALIDATION_ERROR, "Check the highlighted fields.", details)
```

`register_exception_handlers()` 에 한 줄 추가:

```
app.add_exception_handler(RequestValidationError, validation_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)

식	뜻
`exc.errors()`	오류 목록. 칸마다 사전 하나: `{"loc": ("body", "nickname"), "msg": "Field required", "input": …, …}`
`err.get("loc") or ()`	`loc` 가 없거나 비었으면 빈 튜플 `()`
`loc[-1]`	마지막 값. `("body", "nickname")` → `"nickname"`
`str(...)`	목록 칸의 오류는 위치가 숫자(`("body", "tags", 0)` → `0`)라서 글자로 바꿈
`A if 조건 else B`	조건이 참이면 A, 아니면 B (JS 의 `조건 ? A : B`)
`details.setdefault(field, msg)`	`field` 가 아직 없을 때만 넣음 → **칸마다 첫 번째 문구만** 남음

loc	칸 이름
`("body", "email")`	`email`
`("query", "page")`	`page`
`("body",)` (본문 없음)	`body`
`("body", "tags", 0)`	`0`

4. 흐름

1. FastAPI 가 입력 오류 목록을 넘겨준다.
2. 오류마다 `loc` 의 마지막 값을 칸 이름으로, `msg` 를 문구로 꺼낸다.
3. 같은 칸은 첫 문구만 남긴다.
4. `error_json()` 으로 422 응답을 만든다 (01).

5. 왜 이렇게

선택	이유
FastAPI 기본 응답(`{"detail": [...]}`)을 바꿈	기본 모양은 다른 오류와 달라서 `api.js` 가 `error.code` 를 읽지 못한다. 모든 오류를 `{"error": {...}}` 하나로 맞춘다
보낸 값(`input`)은 응답에 넣지 않음	기본 응답은 사용자가 보낸 값을 그대로 돌려준다. 비밀번호 칸이 틀리면 비밀번호가 응답 · 로그에 남는다 (PRD 01 §5). 칸 이름과 문구만 보낸다
칸마다 문구 하나	화면은 칸 아래에 한 줄만 보여 준다. 사전 `{칸: 문구}` 이면 프론트가 `details.email` 로 바로 꺼낸다

선택	이유
칸 이름은 `loc` 의 마지막 값	화면의 입력 칸 이름(`email` · `nickname`)과 같아진다. 앞부분(`body` · `query`)은 화면에 필요 없다
전체 문구는 하나로 고정	칸별 내용은 `details` 에 있다. `message` 는 "표시된 칸을 확인하라"는 안내만

6. 시험 (`tests/test_errors.py` 에 3개 추가)

시험용 앱에 가입과 같은 모양의 스키마(`email` · `password` 8자 이상 · `nickname`)와 `page: int` 를 받는 주소를 만들어 확인했다.

경우	결과
이메일 형식 틀림 + `nickname` 빠짐	422 · `VALIDATION_ERROR` · `details` 의 칸이 `email`, `nickname` 두 개 · `nickname` 은 `"Field required"`
짧은 비밀번호 `Pw!1`	`details` 에 `password` · 응답 어디에도 `Pw!1` 없음
`/search?page=abc`	`details` 에 `page`
본문 없이 POST	422 · `VALIDATION_ERROR`

전체 시험 20개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 03 — `http_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	02 `validation_error_handler()` · 04 `unexpected_error_handler()` (예상 못 한 오류 500)

1. 하는 일

우리 코드가 아니라 **FastAPI · Starlette 가 직접 내는 오류**를 다른 오류와 같은 모양으로 바꾼다.

`GET /api/v1/no-such-endpoint` (없는 API 주소):

	응답
바꾸기 전 (FastAPI 기본)	`404 {"detail": "Not Found"}`
바꾼 뒤	`404 {"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`

2. 언제 쓰나

라우터 코드가 실행되지 않거나, 라이브러리가 스스로 거절할 때 생긴다.

상황	상태	`code`
없는 API 주소 (`/api/v1/projectz`)	404	`NOT_FOUND`
주소는 있는데 방식이 다름 (POST 만 받는 `/auth/login` 에 GET)	405	`METHOD_NOT_ALLOWED`
라이브러리가 요청 자체를 거절 (깨진 요청 등)	400	`BAD_REQUEST`

우리 코드가 일부러 내는 오류는 `AppError` 로 던지고 01 이 처리한다. 이 함수는 그 밖의 기본 오류를 맡는다.

오류 처리기 네 개의 역할

#	처리기	맡는 오류	누가 내나
01	`app_error_handler`	`AppError`	우리 코드 (일부러)
02	`validation_error_handler`	입력 형식 오류	FastAPI (스키마와 비교)
03	**`http_error_handler`**	404 · 405 등	FastAPI · Starlette (주소 · 방식)
04	`unexpected_error_handler`	그 밖의 모든 오류	버그 · DB 끊김 등 (예상 못 함)

페이지 주소는 다르게

없는 **페이지** 주소(`/no-such-page`)는 이 함수까지 오지 않는다. `core/site_files.py` 가 먼저 404 오류 페이지(HTML)를 돌려준다. API 주소(`/api/...`)만 이 함수로 온다.

주소	응답
`/no-such-page`	404 오류 페이지 HTML ("Lost the signal")
`/api/v1/no-such-endpoint`	404 JSON (이 함수)

메시지는 어디에서 보이나

JSON 응답은 사람이 아니라 **페이지의 JS(`api.js`)가 받는다.** 화면에 문구를 보여 줄지는 그 페이지의 JS 가 정한다.

단계	일어나는 일
① 서버	`{"error": {"code": "NOT_FOUND", "message": "Not found."}}`
② `api.js`	`ApiError` 로 바꿈 — `err.code` = `"NOT_FOUND"`, `err.message` = `"Not found."` (JS 02)
③ 페이지 JS	`catch (err)` 로 받아 화면에 표시 (칸 아래 문구 · 알림 · 모달 안 문구). `code` 를 보고 문구를 직접 정할 수도 있다

페이지 JS(P1-07 이후)가 아직 없어서 지금은 화면에서 볼 수 없다. 아래에서 확인한다.

확인 방법	위치
브라우저 개발자 도구	`F12` → Network 탭 → 요청 → Response
API 문서 화면	`http://localhost:8000/docs` → API 선택 → Try it out
주소창에 API 주소 입력	`localhost:8000/api/v1/abc` → JSON 글자가 그대로 보임

오류 HTML 페이지(`frontend/errors/*.html`)와의 관계

요청은 두 종류이고, 오류도 종류에 따라 다르게 돌려준다.

	페이지 요청	API 요청
누가 보내나	**사람** — 주소창 입력 · 링크 클릭	**JS** — `api.get(...)`
주소	`/myPage.html` · `/abc`	`/api/v1/...`
오류 응답	**오류 HTML 페이지** (404 · 500 등 디자인한 화면)	**JSON** (01 ~ 04 처리기)
사용자가 보는 것	오류 페이지 전체 화면	지금 보던 화면 안의 문구

API 오류에 HTML 을 주면 안 된다. 가입 버튼을 눌렀는데 이메일이 중복이라고 전체 화면이 오류 페이지로 바뀌면 안 되기 때문이다. API 는 JSON 으로 답하고, 화면은 그대로 둔 채 문구만 보여 준다.

경우	지금	할 작업
없는 페이지 `/abc`	✅ `errors/404.html` (`site_files.py`)	—
없는 API `/api/v1/abc`	✅ 404 JSON (이 처리기)	—
페이지 요청 중 서버 오류 (500 · 403 등)	❌ JSON 이 나감	**P1-22**: 처리기에서 주소가 `/api` 로 시작하지 않으면 `errors/500.html` 등을 보냄

`errors.py` 의 `ERROR_PAGES` 표(`500 → errors/500.html` …)가 P1-22 에서 쓰는 연결 고리다. 01 ~ 04 처리기에 "API 요청이면 JSON, 페이지 요청이면 HTML" 갈림길만 더하면 된다.

API 오류일 때 JS 가 오류 페이지로 보내는 경우 (2026-10-06 결정)

페이지를 열 때 부르는 **본 데이터**를 못 받으면 그 페이지는 보여 줄 것이 없다. 이때는 JS 가 오류 페이지로 이동한다. 버튼 · 폼 동작의 오류는 화면 안 문구로 보여 준다.

오류가 난 요청	예	처리
본 데이터 — 404	`studioNx.html?projectId=999` → `GET /projects/999/full`	`location.replace('errors/404.html')`
본 데이터 — 403	다른 사람의 프로젝트를 주소로 열기	`location.replace('errors/403.html')`
본 데이터 — 500 이상	대시보드 · 프로젝트를 불러오다 서버 오류	`location.replace('errors/500.html')`
본 데이터 — 401	로그인이 풀림	오류 페이지가 아니라 `requireLogin()` 흐름 (로그인 모달)
버튼 · 폼 동작 (저장 · 가입 · 이름 변경 …)	이메일 중복, 이름 변경 실패	화면 안 문구 — 페이지는 그대로

`replace` 로 이동하는 이유는 JS 07 과 같다 — 뒤로 가기로 오류가 난 페이지에 다시 들어가지 않게 한다.

적용할 곳 (각 함수의 TODO 주석에 적어 둠)	본 데이터
시퀀서 `sequencer.js` `loadProject()`	`GET /projects/{id}/full`
마이페이지 `mypage.js` `loadDashboard()` (P1-17)	`GET /me/dashboard`
회원 정보 `userinfo.js` `loadMe()` (P1-10)	`GET /users/me`
내 프로젝트 `myproject.js` `loadProjects()` (P1-14)	`GET /projects` — 목록이라 404 · 403 은 없고 500 이상만

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다. 이 함수는 **`Request`** · **`JSONResponse`** · **`StarletteHTTPException`** 을 쓴다.

함수

```
HTTP_ERRORS: dict[int, tuple[ErrorCode, str]] = {
    400: (ErrorCode.BAD_REQUEST, "Bad request."),
    401: (ErrorCode.UNAUTHORIZED, "Please sign in."),
    403: (ErrorCode.FORBIDDEN, "You do not have permission."),
    404: (ErrorCode.NOT_FOUND, "Not found."),
    405: (ErrorCode.METHOD_NOT_ALLOWED, "Method not allowed."),
    413: (ErrorCode.FILE_TOO_LARGE, "The file is too large."),
    415: (ErrorCode.UNSUPPORTED_FILE_TYPE, "Unsupported file type."),
    429: (ErrorCode.TOO_MANY_REQUESTS, "Too many requests. Please try again later."),
```

```
}


async def http_error_handler(request: Request, exc: StarletteHTTPException) -> JSONResponse:
    if exc.status_code in HTTP_ERRORS:
        code, message = HTTP_ERRORS[exc.status_code]
    elif exc.status_code >= 500:
        code, message = ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again."
    else:
        code, message = ErrorCode.BAD_REQUEST, "Bad request."
    response = error_json(exc.status_code, code, message)
    if exc.headers:
        response.headers.update(exc.headers)
    return response
```

`register_exception_handlers()` 에 등록 한 줄, `ErrorCode` 에 `BAD_REQUEST` · `METHOD_NOT_ALLOWED` 두 개를 추가했다.

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)
`import A as B`	가져오면서 이름을 `B` 로 바꿈
`HTTPException as StarletteHTTPException`	FastAPI 에도 `HTTPException` 이 있어서, Starlette 쪽임을 이름으로 드러냄

식	뜻
`dict[int, tuple[ErrorCode, str]]`	키는 정수(상태 코드), 값은 `(code, 문구)` 두 개 묶음
`exc.status_code in HTTP_ERRORS`	사전에 그 키가 있는지
`code, message = HTTP_ERRORS[...]`	두 개 묶음을 두 변수에 나눠 담음 (**언패킹**)
`if … elif … else`	순서대로 확인. `elif` 는 JS 의 `else if`
`response.headers.update(exc.headers)`	오류에 딸린 헤더를 응답에 덧붙임
`StarletteHTTPException`	FastAPI 가 쓰는 Starlette 의 `HTTPException`. FastAPI 의 `HTTPException` 은 이것을 물려받으므로 둘 다 이 처리기로 온다

상태	code	message
404	`NOT_FOUND`	`Not found.`
405	`METHOD_NOT_ALLOWED`	`Method not allowed.`
418 (표에 없음, 500 미만)	`BAD_REQUEST`	`Bad request.`
502 (표에 없음, 500 이상)	`INTERNAL_ERROR`	`Something went wrong. Please try again.`

4. 흐름

1. 상태 코드가 표(`HTTP_ERRORS`)에 있으면 표의 `code` · 문구를 쓴다.
2. 없으면 500 이상은 `INTERNAL_ERROR`, 그 밖은 `BAD_REQUEST`.
3. `error_json()` 으로 같은 상태 코드의 응답을 만든다 (01).

4. 오류에 헤더가 있으면 응답에 붙인다.

5. 왜 이렇게

선택	이유
모양을 `{"error": {...}}` 로 맞춤	`api.js` 는 `error.code` 만 읽는다. 기본 모양 `{"detail": …}` 이면 모든 404 가 `INTERNAL_ERROR` 로 보인다 (JS 02 `toApiError`)
문구는 표의 고정 문장	`exc.detail` 은 라이브러리나 다른 코드가 만든 문장이라 내부 정보(서버 주소 등)가 섞일 수 있다. 응답에는 정해 둔 문장만 내보낸다
상태 코드는 그대로	404 는 404 로 둔다. 상태를 바꾸면 브라우저 · 검색 엔진 · 모니터링이 잘못 판단한다
헤더 유지	405 응답에는 받을 수 있는 방식을 알려 주는 `Allow` 헤더가 표준으로 필요하다
`ErrorCode` 두 개 추가	405 · 그 밖의 400 을 나타낼 코드가 없었다. 프론트가 분기할 일은 거의 없지만 `code` 가 비면 안 된다
401 · 403 · 413 · 415 · 429 도 표에 둠	지금은 우리 코드가 `AppError` 로 내지만, 라이브러리가 같은 상태를 내도 같은 `code` 가 나오게 한다

6. 시험 (`tests/test_errors.py` 에 4개 추가)

경우	결과
시험 앱의 없는 주소	404 · 본문이 정확히 `{"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`
GET 만 있는 주소에 POST	405 · `METHOD_NOT_ALLOWED` · `Allow: GET` 헤더 유지
418 · 502 를 내부 문장과 함께 던짐	각각 `BAD_REQUEST` · `INTERNAL_ERROR`, 내부 문장(`10.0.0.5` 등)은 응답에 없음
실제 앱 `/api/v1/no-such-endpoint` · `/no-such-page`	API 는 404 JSON `NOT_FOUND`, 페이지는 404 HTML 그대로

전체 시험 24개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 04 — `unexpected_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식 (처리기 네 개 중 마지막)
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	03 `http_error_handler()` · 05 `decode_access_token()` (토큰 검사)

1. 하는 일

01 ~ 03 이 맡지 않은 **모든 오류**(버그 · DB 끊김 등 예상 못 한 오류)를 500 응답으로 바꾼다. 응답에는 일반 문장만 넣고, 오류 내용은 넣지 않는다.

```
500  {"error": {"code": "INTERNAL_ERROR", "message": "Something went wrong. Please try again.", "details": null}}
```

2. 언제 쓰나

코드가 예상하지 못한 이유로 멈췄을 때다. 정상이라면 일어나지 않아야 하는 오류다.

상황	원래 오류 (서버 안)	응답
코드 버그 (`None` 의 칸을 읽음 등)	`AttributeError: 'NoneType' object has no attribute 'email'`	500 일반 문장
DB 서버가 꺼짐	`OperationalError: (2003, "Can't connect to MySQL server on 'localhost'")`	500 일반 문장
파일 저장 실패	`PermissionError: [Errno 13] … C:\Users\…\uploads\…`	500 일반 문장

오류 내용은 어디에 남나

응답에는 없지만 **사라지지 않는다.** 이 처리기가 응답을 보낸 뒤 Starlette 가 원래 오류를 다시 던지고, 두 곳이 그것을 기록한다.

순서	일어나는 일
① 라우터에서 오류	`RuntimeError(...)`
② 이 처리기	사용자에게 500 일반 문장 응답
③ Starlette	응답을 보낸 뒤 **원래 오류를 다시 던짐** (`raise exc`)
④ uvicorn	서버를 실행한 터미널에 오류 전체(어느 파일 몇째 줄인지) 출력
⑤ Sentry	`SENTRY_DSN` 이 설정돼 있으면 Sentry 에 보냄 (`core/sentry.py` 가 비밀번호 · 토큰 · 쿠키를 지운 뒤)

누가 보나	보는 것
사용자 (화면)	"Something went wrong. Please try again."
개발자	터미널 로그 · Sentry — 오류 종류 · 내용 · 위치 전체

처리기 네 개 정리 (P1-03 오류 응답 형식 완료)

#	처리기	맡는 오류	상태
01	`app_error_handler`	우리 코드가 일부러 던진 `AppError`	오류에 적힌 값
02	`validation_error_handler`	입력 형식 오류	422
03	`http_error_handler`	없는 주소 · 다른 방식 등	404 · 405 …
04	**`unexpected_error_handler`**	그 밖의 모든 오류	500

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다. 이 함수는 **`Request`** · **`JSONResponse`** 만 쓴다. `Exception` 은 파이썬 기본이라 import 가 필요 없다.

함수

```
async def unexpected_error_handler(request: Request, exc: Exception) -> JSONResponse:
    return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
```

`register_exception_handlers()` 에 등록:

```
app.add_exception_handler(Exception, unexpected_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)

식	뜻
`exc: Exception`	`Exception` 은 거의 모든 오류의 부모. 그래서 01 ~ 03 이 맡지 않은 오류가 모두 여기로 온다
`exc` 를 쓰지 않음	일부러다. 오류 내용을 응답에 넣지 않으므로 받기만 한다
`add_exception_handler(Exception, …)`	FastAPI 는 `Exception` 처리기를 다른 처리기와 따로, **가장 바깥**(`ServerErrorMiddleware`)에 둔다. 더 구체적인 처리기(01 ~ 03)가 먼저 잡고, 아무도 잡지 않은 오류만 여기로 온다

4. 흐름

1. 01 ~ 03 이 잡지 않은 오류가 가장 바깥까지 올라온다.

2. 이 처리기가 500 일반 문장 응답을 만든다.
3. Starlette 가 응답을 보낸 뒤 원래 오류를 다시 던진다 → 터미널 로그 · Sentry 가 기록.

5. 왜 이렇게

선택	이유
오류 내용을 응답에 넣지 않음	오류 문장에는 SQL · 파일 경로 · DB 주소 · 비밀값이 섞일 수 있다 (위 §2 의 예). 공격자에게 서버 구조를 알려 주게 된다 (PRD 01 §5)
기록 코드를 따로 쓰지 않음	Starlette 가 오류를 다시 던져 uvicorn · Sentry 가 이미 기록한다. 여기서 또 기록하면 Sentry 에 같은 오류가 두 번 쌓인다
모양은 다른 오류와 같게	`api.js` 가 `INTERNAL_ERROR` 로 읽는다. 기본 500 응답은 `Internal Server Error` 글자(JSON 아님)라서 프론트가 형식을 알 수 없다
문구는 "다시 시도"	DB 가 잠깐 끊긴 경우처럼 다시 하면 되는 일이 많다. 계속되면 Sentry 에서 알 수 있다

6. 시험 (`tests/test_errors.py` 에 2개 추가)

시험용 주소가 `RuntimeError("SELECT * FROM users WHERE password_hash='…'")` 를 던지게 했다.

경우	결과
응답 확인 (`raise_server_exceptions=False`)	500 · 본문이 정확히 위 JSON · 응답에 `password_hash` · `RuntimeError` 없음
다시 던지는지 확인 (기본 설정)	시험 도구에서 `RuntimeError` 가 그대로 올라옴 → 로그 · Sentry 가 기록할 수 있음

`raise_server_exceptions=False`: 시험 도구(`TestClient`)는 기본으로 서버 오류를 다시 던져 시험을 멈춘다. 실제 응답을 보려고 이 옵션을 끈다.

전체 시험 26개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 05 — `decode_access_token()` · `_decode_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-03 백엔드 공통 — 토큰 검사 · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	04 `unexpected_error_handler()` · 06 `get_current_user()`

1. 하는 일

요청에 실려 온 Access Token 이 **우리 서버가 만든 진짜 토큰이고, 아직 유효한지** 확인한 뒤, 토큰 주인의 `user_id` 를 돌려준다. 문제가 있으면 401 오류를 던진다.

`decode_access_token("eyJhbGciOi…")` → `7` (회원 번호)

함수	역할
`decode_access_token()`	Access Token 검사. 바깥에서 부르는 이름
`_decode_token()`	실제 검사. P1-05 의 `decode_refresh_token()` 도 같은 함수를 쓴다 (`type` 만 다름)

2. 언제 쓰나

로그인이 필요한 **모든 API 요청마다** 한 번씩 불린다.

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard` + 헤더 `Authorization: Bearer eyJhbGciOi…` (JS 02)
② `get_current_user()` (다음 06)	헤더에서 토큰 글자를 꺼내 **이 함수**에 넘김
③ 이 함수	검사 → `7`
④ `get_current_user()`	DB 에서 7번 회원을 찾아 라우터에 넘김
실패하면	401 `UNAUTHORIZED` → 브라우저의 `api.js` 가 리프레시 쿠키로 새 토큰을 받고 한 번 다시 보냄

JWT(토큰)의 생김새

토큰은 점(`.`)으로 나뉜 세 부분이다.

부분	예	내용
머리	`eyJhbGciOiJIUzI1NiJ9`	서명 방식 `{"alg": "HS256"}`
내용	`eyJzdWIiOiI3Ii…`	`{"sub": "7", "type": "access", "exp": 1791273600}`
서명	`kQ3x…`	머리 + 내용을 **서버 비밀값(`JWT_SECRET`)**으로 계산한 값

칸	뜻
`sub`	토큰 주인 — 회원 번호 (글자로 저장)

칸	뜻
`type`	`access` (30분, API 요청용) 또는 `refresh` (3일, 재발급용)
`exp`	만료 시각 (1970년부터 센 초)

내용은 암호화되지 않는다. 누구나 풀어서 읽을 수 있다. 그래서 비밀번호 같은 것은 넣지 않는다. 대신 **서명** 덕분에 내용을 바꿀 수 없다 — `sub` 를 `7` 에서 `1` 로 바꾸면 서명이 맞지 않고, 비밀값을 모르는 사람은 맞는 서명을 만들 수 없다.

3. 코드

import (파일 맨 위)

```
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다.

import	가져오는 것	쓰는 곳
`jwt`	PyJWT 라이브러리 (`requirements.txt` 의 `pyjwt`)	`jwt.decode` · `jwt.PyJWTError`
`get_settings`	`.env` 를 읽은 설정 (`app/core/config.py`)	`get_settings().jwt_secret`
`AppError` · `ErrorCode`	01 의 오류 클래스 · 코드 (`app/core/errors.py`)	401 `UNAUTHORIZED`

설치 이름(`pyjwt`)과 import 이름(`jwt`)이 다르다.

함수

```
JWT_ALGORITHM = "HS256"


def decode_access_token(token: str) -> int:
    return _decode_token(token, "access")


def _decode_token(token: str, expected_type: str) -> int:
    invalid = AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    try:
        payload = jwt.decode(
            token,
            get_settings().jwt_secret,
            algorithms=[JWT_ALGORITHM],
            options={"require": ["exp", "sub", "type"]},
        )
    except jwt.PyJWTError:
        raise invalid from None
    if payload["type"] != expected_type:
        raise invalid
    try:
        return int(payload["sub"])
```

```
    except (TypeError, ValueError):
        raise invalid from None
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `jwt.decode`)
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)
`app.…`	우리 프로젝트(`studionx/backend/app/`)의 파일. `app.core.errors` = `app/core/errors.py`

식	뜻
`jwt.decode(token, 비밀값, algorithms=[…])`	PyJWT 라이브러리. 서명 확인 + 만료(`exp`) 확인 후 내용을 사전으로 돌려줌. 하나라도 틀리면 오류
`options={"require": [...]}`	이 칸들이 없으면 실패
`except jwt.PyJWTError:`	PyJWT 의 모든 오류(만료 `ExpiredSignatureError`, 서명 틀림 `InvalidSignatureError`, 형식 틀림 `DecodeError` …)의 부모. 한 번에 잡는다
`raise invalid from None`	우리 오류로 바꿔 던짐. `from None` 은 "원래 오류는 이어 붙이지 말라"는 뜻 — 로그가 짧아진다
`int(payload["sub"])`	`"7"` → `7`. `"admin"` 이면 `ValueError`, 숫자가 아닌 종류면 `TypeError`
`_decode_token` 의 `_`	이 파일 안에서만 쓰는 함수라는 파이썬 관례

토큰	결과
정상 (`sub: "7"`, `type: access`, 만료 전)	`7`
만료됨	401
다른 비밀값으로 서명	401
내용만 바꿈 (`sub` 7 → 1)	401 (서명 불일치)
`type: refresh`	401
`sub` 또는 `type` · `exp` 없음	401
`sub: "admin"`	401
서명 없는 토큰 (`alg: none`)	401
`"not-a-token"` · `""`	401

4. 흐름

1. 서명 · 만료 · 필수 칸을 확인한다 (`jwt.decode`). 실패하면 401.
2. `type` 이 `access` 인지 확인한다. 아니면 401.
3. `sub` 를 정수로 바꿔 돌려준다. 안 되면 401.

5. 왜 이렇게 (보안)

선택	이유
`algorithms` 를 `HS256` 하나로 고정	토큰 머리의 `alg` 는 보낸 사람이 마음대로 쓸 수 있다. `"none"`(서명 없음)을 믿으면 누구나 `sub: 1` 토큰을 만들 수 있다. 서버가 정한 방식만 받는다

선택	이유
`type` 확인	리프레시 토큰(3일)을 Access Token 자리에 넣어 30분 제한을 피하지 못하게 한다
실패 이유와 상관없이 같은 401 · 같은 문구	만료인지 위조인지 알려 주면 공격자에게 힌트가 된다. 프론트는 어차피 같은 처리(재발급)를 한다
`"Please sign in again."` 의 `UNAUTHORIZED`	프론트 `api.js` 가 재발급을 시도하는 조건과 같은 코드 (JS 02)
검사 부분을 `_decode_token()` 으로 분리	P1-05 `decode_refresh_token()` 은 `return _decode_token(token, "refresh")` 한 줄이 된다
비밀값은 `.env` 의 `JWT_SECRET`	코드 · Git 에 넣지 않는다. 32바이트 이상이어야 한다 (짧으면 PyJWT 가 경고). 지금 로컬 값은 64바이트 — 값은 출력하지 않고 길이만 확인

비밀값이 새면 누구나 아무 회원의 토큰을 만들 수 있다. 그때는 `JWT_SECRET` 을 바꾸면 기존 토큰이 모두 무효가 된다 (모든 사용자가 다시 로그인).

6. 시험 (`tests/test_security.py`, 새로 만듦 · 9개)

토큰 발급(`create_access_token`)은 P1-05 라서 시험에서 `jwt.encode` 로 직접 만들었다.

경우	결과
정상 토큰	`7`
만료 · 다른 비밀값 · `type: refresh`	401 `UNAUTHORIZED` "Please sign in again."
`sub` · `type` · `exp` 중 하나 없음 (3개)	401
`sub: "admin"`	401
`"not-a-token"` · 빈 글자	401
서명 없는 토큰 (`alg: none`)	401
내용만 바꾼 토큰	401

전체 시험 35개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 06 — `get_current_user()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통 — 로그인한 사용자 찾기 · PRD 01 FR-105 · FR-108
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	05 `decode_access_token()` · 07 `get_current_user_optional()`

1. 하는 일

요청 헤더의 Access Token 으로 **요청한 회원을 DB 에서 찾아** 라우터에 넘긴다. 토큰이 없거나, 틀렸거나, 회원이 활동 중이 아니면 401 오류를 던진다.

`Authorization: Bearer eyJhbGciOi…` → `User(user_id=7, …)`

2. 언제 쓰나

로그인이 필요한 API 가 함수 인자에 `user: CurrentUser` 라고 적으면, FastAPI 가 라우터보다 **먼저** 이 함수를 부른다. 라우터는 검사가 끝난 회원만 받는다.

```
@router.get("/users/me")
def read_me(user: CurrentUser) -> MeResponse:   # user = 이 함수가 돌려준 회원
    ...
```

이 함수를 쓰는 API (지금 뼈대 기준)	화면
`GET /users/me` · `PATCH /users/me` · 프로필 사진 · 비밀번호 변경 · 탈퇴	userInfo.html
`GET /me/dashboard`	myPage.html
`GET · POST · PATCH · DELETE /projects…`	myProject.html · studioNx.html
쓰지 않음: `POST /inquiries`	로그인 선택 → 07 `get_current_user_optional()`

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard` + `Authorization: Bearer eyJ…` (JS 02)
② `bearer_scheme`	헤더에서 `eyJ…` 만 꺼냄
③ `get_db`	DB 세션을 엶
④ **이 함수**	토큰 검사(05) → 7 → DB 에서 7번 회원 → 상태 확인
⑤ 라우터	`user` 로 7번 회원을 받아 대시보드를 만듦
실패하면	라우터는 실행되지 않음. 401 → 브라우저 `api.js` 가 재발급 후 한 번 다시 보냄

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import 이다. **`AppError` · `ErrorCode` · `decode_access_token` 두 줄을 이번에 추가했다.**

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 덧붙임 정보를 다는 도구	`Bearer` · `Db` · `CurrentUser` 별칭
`Depends`	"이 값은 이 함수로 만들어 넣어라" 표시	별칭 안
`HTTPBearer` · `HTTPAuthorizationCredentials`	`Authorization: Bearer …` 헤더 읽기 · 그 결과	`bearer_scheme` · `credentials`
`Session`	DB 세션 타입 (SQLAlchemy)	`Db`
`AppError` · `ErrorCode`	01 의 오류 클래스 · 코드	401
`decode_access_token`	05 의 토큰 검사	2단계
`get_db`	요청마다 DB 세션을 열고 닫는 함수	`Db`
`User`	users 테이블 모델	`db.get(User, …)`

함수 (이미 있던 별칭 포함)

```
bearer_scheme = HTTPBearer(auto_error=False)
Bearer = Annotated[HTTPAuthorizationCredentials | None, Depends(bearer_scheme)]
Db = Annotated[Session, Depends(get_db)]


def get_current_user(
    credentials: Bearer,
    db: Db,
) -> User:
    if credentials is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_access_token(credentials.credentials)
    user = db.get(User, user_id)
    if user is None or user.status != "ACTIVE":
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user


CurrentUser = Annotated[User, Depends(get_current_user)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`app.…`	우리 프로젝트(`studionx/backend/app/`)의 파일. `app.api.deps` = `app/api/deps.py`

식	뜻
`Depends(함수)`	FastAPI 가 요청마다 그 함수를 먼저 실행해 결과를 인자로 넣어 줌 ("의존성")
`Annotated[타입, Depends(…)]`	타입과 `Depends` 를 한 이름으로 묶음. `user: CurrentUser` 한 번만 쓰면 됨
`HTTPBearer(auto_error=False)`	헤더가 없거나 `Bearer` 로 시작하지 않으면 **오류 대신** **`None`** 을 넣어 줌. 기본값(`True`)이면 FastAPI 가 그 자리에서 401 오류를 던짐
`credentials.credentials`	`Bearer eyJ…` 에서 `eyJ…` 부분 (`credentials.scheme` 은 `"Bearer"`)
`db.get(User, 7)`	기본 키(`user_id`)가 7 인 행. 없으면 `None`
`A is None or B`	`A` 가 참이면 `B` 를 보지 않음 — `None.status` 오류가 나지 않음

요청	결과
헤더 없음	401 "Please sign in."
`Authorization: Basic …`	401 "Please sign in." (`Bearer` 가 아님 → `None`)
토큰이 틀림 · 만료	401 "Please sign in again." (05 가 던짐)
토큰은 맞는데 회원 없음	401 "Please sign in again."
회원 `status` 가 `SUSPENDED` · `DELETED`	401 "Please sign in again."
회원 `status` 가 `ACTIVE`	그 회원 (`User`)

4. 흐름

1. 헤더가 없으면 401 "Please sign in."
2. 토큰을 검사해 `user_id` 를 얻는다 (05). 실패하면 401.
3. DB 에서 그 회원을 찾는다.
4. 없거나 `ACTIVE` 가 아니면 401 "Please sign in again."
5. 회원을 돌려준다.

5. 왜 이렇게

선택	이유
토큰만 믿지 않고 DB 를 한 번 더 확인	Access Token 은 30분 동안 유효하다. 그 사이 탈퇴(`DELETED`)하거나 정지(`SUSPENDED`)된 회원의 토큰을 막는다 (FR-108)
없는 회원 · 정지 · 탈퇴를 같은 401 · 같은 문구로	05 와 같은 이유. 이유를 알려 주면 "이 번호의 회원은 탈퇴했다" 같은 정보가 샌다
403 이 아니라 401	프론트 `api.js` 는 401 `UNAUTHORIZED` 일 때만 재발급 → 실패하면 로그인 화면으로 보낸다 (JS 02). 정지 · 탈퇴 회원은 재발급도 거절되어(P1-05) 결국 로그인 화면으로 간다
문구 두 가지	헤더 없음은 처음부터 로그인 안 한 것 → "Please sign in." 토큰이 있는데 안 됨 → "Please sign in again."

선택	이유
`auto_error=False` 로 직접 401	같은 `Bearer` 를 07 `get_current_user_optional()` 도 쓴다. 07 은 헤더가 없을 때 오류가 아니라 `None`(손님)으로 처리해야 하므로, 헤더 읽기는 오류를 내지 않게 두고 각 함수가 판단한다
`status != "ACTIVE"` 를 글자로 비교	users 모델의 `status` 칸은 이 작업의 다음 순서(모델 컬럼)에서 만든다. 그때 `StrEnum` 으로 만들면 `UserStatus.ACTIVE == "ACTIVE"` 가 참이라 이 코드는 그대로 동작한다

6. 시험 (`tests/test_deps.py`, 새로 만듦 · 7개)

users 모델의 컬럼(`status` 등)은 아직 없어서, 진짜 DB 대신 **가짜 세션(`FakeDb`)**을 넣었다. `FakeDb` 는 `db.get(User, id)` 만 흉내 내고, 받은 인자를 기록한다. 시험용 앱에 `GET /whoami` (`user: CurrentUser` → `{"user_id": …}`)를 만들어 요청했다.

경우	결과
정상 토큰 + `ACTIVE` 회원 7	200 `{"user_id": 7}` · DB 에 `(User, 7)` 로 한 번 물어봄
헤더 없음	401 "Please sign in." · DB 에 묻지 않음
`Authorization: Basic …`	401 "Please sign in."
틀린 토큰	401 "Please sign in again." · DB 에 묻지 않음
토큰 회원 99 가 DB 에 없음	401 "Please sign in again."
`SUSPENDED` · `DELETED` 회원 (2개)	401 "Please sign in again."

`app.dependency_overrides[get_db] = …`: 시험에서 `get_db` 대신 다른 함수를 쓰게 하는 FastAPI 기능이다.

전체 시험 42개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 07 — `get_current_user_optional()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통 — 로그인이 선택인 API · PRD 01 FR-112
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	06 `get_current_user()` · 08 `User` 모델

1. 하는 일

로그인한 회원이면 그 회원을, 아니면 `None`(손님)을 라우터에 넘긴다. **401 오류를 내지 않는다.** 06 과 검사는 같고, 실패했을 때의 처리만 다르다.

	06 `get_current_user()`	07 이 함수
로그인한 `ACTIVE` 회원	회원	회원
헤더 없음 · 토큰 틀림 · 회원 없음 · 정지 · 탈퇴	401 오류 (라우터 실행 안 됨)	`None` (라우터는 손님으로 실행)
라우터에 적는 이름	`user: CurrentUser`	`user: OptionalUser`

2. 언제 쓰나

지금 쓰는 곳은 **문의 등록** `POST /api/v1/inquiries` 하나다 (home.html 아래쪽 Contact 폼).

보내는 사람	이 함수의 결과	저장되는 문의 (`inquiries` 테이블)
로그인 안 한 방문자	`None`	`user_id = NULL` — 폼에 적은 이메일로만 답장
로그인한 회원 7	7번 회원	`user_id = 7` — 관리자가 어느 회원의 문의인지 알 수 있음
토큰이 만료된 회원	`None`	`user_id = NULL` — 문의는 그대로 접수됨

```
def create_inquiry(data: InquiryCreateRequest, request: Request, user: OptionalUser, db: Db):
    ...   # P1-12: user 가 있으면 inquiry.user_id = user.user_id
```

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
```

```
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import 이다 (06 과 같음). 이 함수는 **`AppError`** 만 직접 쓴다. 나머지 검사는 06 `get_current_user()` 를 불러서 한다.

함수

```
def get_current_user_optional(
    credentials: Bearer,
    db: Db,
) -> User | None:
    if credentials is None:
        return None
    try:
        return get_current_user(credentials, db)
    except AppError:
        return None


OptionalUser = Annotated[User | None, Depends(get_current_user_optional)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`-> User | None`	회원 또는 `None` 을 돌려준다는 표시
`get_current_user(credentials, db)`	06 을 **보통 함수로 직접** 부름. `Depends` 를 거치지 않으므로 이미 받은 `credentials` · `db` 를 그대로 넘김
`try: … except AppError: return None`	06 이 던진 401(`AppError`)을 잡아 `None` 으로 바꿈. 05 의 토큰 오류도 `AppError` 라서 함께 잡힘
`except AppError` (모든 오류가 아님)	DB 가 끊긴 오류 같은 것은 잡지 않음 → 04 가 500 으로 처리

4. 흐름

1. 헤더가 없으면 `None`. DB 에 묻지 않는다.
2. 있으면 06 과 같은 검사를 한다.
3. 통과하면 회원, 401 이면 `None`.

5. 왜 이렇게

선택	이유
실패해도 401 을 내지 않음	문의는 손님도 보낼 수 있다. 만료된 토큰 때문에 문의가 거절되면 안 된다
검사를 06 에 맡김	검사 규칙(토큰 · 회원 없음 · 정지 · 탈퇴)이 한 곳에만 있다. 06 을 고치면 07 도 같이 바뀐다

선택	이유
`AppError` 만 잡음	DB 오류까지 `None` 으로 바꾸면 서버 고장이 숨는다. 고장은 04 가 500 으로 알리고 Sentry 에 기록한다
정지 · 탈퇴 회원도 `None`	그 회원 이름으로 연결하지 않고 손님 문의로 받는다

P1-12 에서 알아 둘 것: Access Token 은 메모리에만 있어서 home.html 을 새로 열면 비어 있다 (JS 01). 로그인한 회원의 문의를 회원과 연결하려면, `home.js` 가 보내기 전에 `isLoggedIn()` (JS 06)으로 토큰을 먼저 받아 두어야 한다.

6. 시험 (`tests/test_deps.py` 에 6개 추가 · 모두 13개)

06 과 같은 가짜 세션(`FakeDb`)을 쓰고, 시험용 앱에 `GET /maybe` (`user: OptionalUser` → `{"user_id": 7 또는 null}`)를 만들었다.

경우	결과
정상 토큰 + `ACTIVE` 회원 7	200 `{"user_id": 7}`
헤더 없음	200 `{"user_id": null}` · DB 에 묻지 않음
틀린 토큰	200 `{"user_id": null}`
토큰 회원 99 가 DB 에 없음	200 `{"user_id": null}`
`DELETED` 회원	200 `{"user_id": null}`
`Authorization: Basic …`	200 `{"user_id": null}`

전체 시험 48개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 08 — `User` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/user.py`
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	07 `get_current_user_optional()` · 09 `Inquiry` 모델

1. 하는 일

MySQL 의 `users` 테이블을 **파이썬 클래스** `User` 로 옮겨 적는다. 그러면 SQL 문 대신 파이썬 코드로 회원을 읽고 쓸 수 있다.

SQL	파이썬 (이 클래스가 있으면)
`SELECT * FROM users WHERE user_id = 7`	`db.get(User, 7)`
`user.status` 칸 읽기	`user.status` → `UserStatus.ACTIVE`
`INSERT INTO users (email, …) VALUES (…)`	`db.add(User(email=…, …))`

테이블은 이 클래스가 만들지 않는다. 테이블은 `studionx/db/01_phase1_account.sql` 로 이미 만들어져 있고, 이 클래스는 그 모양을 그대로 따라 적은 것이다. 지금까지는 `user_id` 한 칸만 있었고, 이번에 나머지 14칸을 채웠다.

2. 언제 쓰나

회원을 다루는 모든 백엔드 코드가 이 클래스를 쓴다.

쓰는 곳	읽거나 쓰는 칸
06 `get_current_user()`	`status` — `ACTIVE` 인지
P1-04 회원가입	`email` · `password_hash` · `nickname` · `privacy_agreed_at` · `privacy_version` 넣기
P1-05 로그인	`email` 로 찾기 · `password_hash` 비교 · `last_login_at` 기록
P1-08 회원 정보 (userInfo.html)	`nickname` · `profile_image_path` 수정, 탈퇴 시 `status = DELETED` · `deleted_at`
P1-16 대시보드 (myPage.html)	`storage_quota_bytes` · `storage_used_bytes` — 저장 공간 막대
P1-21 관리자 화면	`role` — `ADMIN` 만 들어감

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, String, text
```

```
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다. 모두 이 클래스에서 쓴다.

import	가져오는 것	쓰는 곳
`datetime` (파이썬 기본)	날짜 · 시각 타입	`created_at` 등 시각 칸
`StrEnum` (파이썬 기본)	값이 글자인 열거형	`UserRole` · `UserStatus`
`BigInteger` · `String`	SQL 의 `BIGINT` · `VARCHAR(n)`	숫자 · 글자 칸
`Enum`	SQL 의 `ENUM(…)`	`role` · `status`
`text`	SQL 글자를 그대로 넘김	기본값 `CURRENT_TIMESTAMP(3)` 등
`FetchedValue`	"DB 가 값을 바꾼다" 표시	`updated_at`
`DATETIME` (MySQL 전용)	`DATETIME(3)` — 밀리초까지	시각 칸
`Mapped` · `mapped_column`	칸 선언	모든 칸
`Base`	모든 모델의 부모 (`app/db/session.py`)	`class User(Base)`

함수 (이 작업은 함수가 아니라 클래스 · 칸 선언이다)

```
class UserRole(StrEnum):
    USER = "USER"
    ADMIN = "ADMIN"


class UserStatus(StrEnum):
    ACTIVE = "ACTIVE"
    SUSPENDED = "SUSPENDED"
    DELETED = "DELETED"


class User(Base):
    __tablename__ = "users"

    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    email: Mapped[str] = mapped_column(String(255), unique=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    nickname: Mapped[str] = mapped_column(String(50), unique=True)
    profile_image_path: Mapped[str | None] = mapped_column(String(500))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole), server_default=UserRole.USER)
    status: Mapped[UserStatus] = mapped_column(Enum(UserStatus), server_default=UserStatus.ACTIVE)
    last_login_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    privacy_version: Mapped[str] = mapped_column(String(20))
    storage_quota_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("5368709120"))
    storage_used_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
```

```
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL 과 한 줄씩 비교

SQL (01_phase1_account.sql)	파이썬
`email VARCHAR(255) NOT NULL` + `UNIQUE KEY uq_users_email`	`Mapped[str]` · `String(255)` · `unique=True`
`profile_image_path VARCHAR(500) NULL`	`Mapped[str | None]`
`role ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER'`	`Enum(UserRole)` · `server_default=UserRole.USER`
`last_login_at DATETIME(3) NULL`	`Mapped[datetime | None]` · `DATETIME(fsp=3)`
`storage_quota_bytes BIGINT UNSIGNED NOT NULL DEFAULT 5368709120`	`BigInteger` · `server_default=text("5368709120")` (5GB)
`updated_at … DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)`	`server_default=text(…)` · `server_onupdate=FetchedValue()`

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안의 **MySQL 전용** 타입

식	뜻
`Mapped[str]`	이 칸의 파이썬 타입. **`None` 이 없으면 `NOT NULL`**, `Mapped[str | None]` 이면 `NULL` 허용
`mapped_column(String(255))`	SQL 타입 `VARCHAR(255)`
`unique=True`	같은 값을 두 번 넣을 수 없음 (SQL 의 `UNIQUE KEY`)
`class UserStatus(StrEnum)`	정해진 값만 쓰게 하는 목록. `UserStatus.ACTIVE == "ACTIVE"` 가 참
`server_default=…`	값을 안 넣으면 **DB 가** 채우는 기본값. 파이썬은 넣지 않고 DB 에 맡김
`text("CURRENT_TIMESTAMP(3)")`	글자 `'CURRENT_TIMESTAMP(3)'` 가 아니라 SQL 식으로 넘김
`server_onupdate=FetchedValue()`	"행을 고치면 DB 가 이 칸을 바꾼다"는 표시. 저장 뒤 새 값을 다시 읽어 오게 함
`DATETIME(fsp=3)`	`DATETIME(3)`. `fsp` = 초 아래 자릿수 (3 = 밀리초)

4. 흐름 (회원 하나를 넣을 때)

1. `User(email=…, password_hash=…, nickname=…, privacy_agreed_at=…, privacy_version=…)` 를 만든다. 다른 칸은 비워 둔다.
2. `db.add(user)` → `db.flush()` 로 DB 에 `INSERT` 한다.
3. DB 가 `user_id` · `role=USER` · `status=ACTIVE` · `storage_quota_bytes=5GB` · `created_at` 을 채운다.
4. `db.refresh(user)` 로 채워진 값을 다시 읽으면 `user.status` 는 `UserStatus.ACTIVE` 이다.

5. 왜 이렇게

선택	이유
기본값은 `server_default` (DB 가 채움)	기본값의 원본은 SQL 파일 하나다. 파이썬에도 따로 적으면 둘이 달라질 수 있다. SQLAdmin 이나 DB 도구로 직접 넣어도 같은 값이 들어간다
`role` · `status` 를 `StrEnum` 으로	오타(`"ACTIVEE"`)를 막고, 편집기가 값 목록을 보여 준다. 06 의 `user.status != "ACTIVE"` 도 그대로 동작한다
시각 칸은 `DATETIME(3)` (시간대 없음)	DB 에는 UTC 로 저장하기로 했다 (`docs/DB.md`). 읽은 값에는 시간대 정보가 없으니 UTC 로 보고 쓴다
`BIGINT UNSIGNED` 를 `BigInteger` 로	테이블은 SQL 파일이 만들므로 부호 여부는 DB 가 지킨다. 파이썬에서 읽고 쓰는 데는 차이가 없고, 다른 모델과도 같다
관계(`relationship`)는 아직 안 넣음	`user.projects` 처럼 쓸 곳이 생길 때(P1-09 등) 넣는다. 지금 넣으면 쓰지 않는 코드가 된다

6. 시험 (`tests/test_user_model.py`, 새로 만듦 · 4개)

실제 MySQL 의 `users` 테이블과 비교했다. DB 에 연결할 수 없으면 건너뛴다. 넣은 행은 시험이 끝나면 되돌려서 (rollback) DB 에 남지 않는다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 이름 15개 · 순서 · `NULL` 허용 여부가 실제 테이블과 같음
중복 금지 칸	`email` · `nickname` 두 칸
넣으면 DB 가 기본값을 채우는지	5칸만 넣고 저장 → `role` `USER` · `status` `ACTIVE` · 저장 공간 5GB · 사용량 0 · `created_at` 채워짐 · `last_login_at` · `deleted_at` 비어 있음
06 `get_current_user()` 를 실제 DB 로	`ACTIVE` 회원 → 그 회원, `DELETED` 회원 → 401

06 · 07 시험은 가짜 DB 세션으로 했는데, 이번에 칸이 생겨서 실제 DB 로도 확인했다.

전체 시험 52개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 09 — `Inquiry` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/inquiry.py`
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	08 `User` 모델 · 10 `Project` 모델

1. 하는 일

MySQL 의 `inquiries` 테이블(문의)을 파이썬 클래스 `Inquiry` 로 옮겨 적는다. 08 `User` 와 같은 방식이고, 이번에 새로 나온 것은 **다른 테이블을 가리키는 칸(FK)** 과 **긴 글 칸(`TEXT`)** 이다.

새로 나온 것	칸	뜻
FK (외래 키)	`user_id` → `users.user_id`	이 문의를 보낸 회원. 손님이면 비어 있음
`TEXT`	`content` · `answer`	길이 제한이 거의 없는 글 (`VARCHAR` 는 길이를 정해야 함)

2. 언제 쓰나

쓰는 곳	하는 일	칸
P1-12 문의 등록 (home.html Contact 폼)	문의 저장	`user_id`(07 의 결과) · `email` · `title` · `content` · `privacy_agreed_at`
P1-21 관리자 화면 (SQLAdmin)	목록 보기 · 답변	`status` · `answer` · `answered_at`

문의 상태 (`status`)	뜻
`RECEIVED` (기본)	접수됨, 아직 답변 없음
`ANSWERED`	관리자가 답변함
`CLOSED`	처리 끝

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, ForeignKey, String, Text, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다. 08 과 같고, **`ForeignKey`** · **`Text`** 두 개가 새로 들어갔다.

import	가져오는 것	쓰는 곳
`ForeignKey`	다른 테이블의 칸을 가리키는 표시	`user_id`
`Text`	SQL 의 `TEXT`	`content` · `answer`
나머지	08 의 표 와 같음	

함수 (클래스 · 칸 선언)

```
class InquiryStatus(StrEnum):
    RECEIVED = "RECEIVED"
    ANSWERED = "ANSWERED"
    CLOSED = "CLOSED"


class Inquiry(Base):
    __tablename__ = "inquiries"

    inquiry_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    user_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    name: Mapped[str | None] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(255))
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    status: Mapped[InquiryStatus] = mapped_column(
        Enum(InquiryStatus), server_default=InquiryStatus.RECEIVED
    )
    answer: Mapped[str | None] = mapped_column(Text)
    answered_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

SQL 과 비교 (08 에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`user_id BIGINT UNSIGNED NULL` + `FOREIGN KEY (user_id) REFERENCES users (user_id)`	`Mapped[int | None]` · `ForeignKey("users.user_id")`
`content TEXT NOT NULL`	`Mapped[str]` · `Text`
`KEY idx_inquiries_status (status)` (검색용 색인)	적지 않음 — 아래 §5

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`ForeignKey("users.user_id")`	"이 칸의 값은 `users` 테이블 `user_id` 중 하나"라는 표시. `"테이블.칸"` 글자로 적음
`Mapped[int | None]` + FK	비어 있어도 됨(손님 문의). 값이 있으면 실제 회원 번호여야 함
`Text`	길이를 적지 않음. MySQL `TEXT` 는 약 64KB (한글 약 2만 자)

4. 흐름 (손님 문의 하나를 넣을 때)

1. `Inquiry(email=…, title=…, content=…, privacy_agreed_at=…)` — 4칸만 넣는다.
2. `db.add` → `db.flush()` 로 `INSERT`.
3. DB 가 `inquiry_id` · `status=RECEIVED` · `created_at` 을 채운다. `user_id` 는 비어 있다.
4. 로그인 회원이면 1에서 `user_id=7` 을 함께 넣는다. 없는 번호면 DB 가 거절한다.

5. 왜 이렇게

선택	이유
`user_id` 를 비울 수 있게	손님도 문의할 수 있다 (FR-112). 07 이 `None` 을 주면 그대로 비워 둔다
FK 를 파이썬에도 적음	테이블은 SQL 이 만들지만, 적어 두면 나중에 `relationship`(문의 → 회원)을 붙일 수 있고 코드만 봐도 연결을 알 수 있다
색인(`idx_inquiries_status`)은 적지 않음	색인은 DB 가 검색을 빠르게 하려고 가진 것이다. 파이썬에서 읽고 쓰는 데 필요 없다. 08 도 같다
`content` 최대 2,000자는 여기서 검사하지 않음	칸(`TEXT`)은 2,000자보다 훨씬 크다. 글자 수 검사는 입력 스키마(P1-12, 02 의 422)가 한다
`name` 칸도 적음	폼에서 받지 않는 예비 칸이지만 테이블에 있으므로 모델도 같게 맞춘다
회원 탈퇴 때 문의는 남김	FK 에 `ON DELETE` 가 없다. 탈퇴는 행을 지우지 않고 `status=DELETED` 로 바꾸므로(08) 문의의 `user_id` 도 그대로 남는다

6. 시험

`tests/test_inquiry_model.py` (새로 만듦 · 5개) — 실제 MySQL 과 비교했다. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 12개 이름 · 순서 · `NULL` 허용 · FK(`user_id` → `users.user_id`)가 실제 테이블과 같음
손님 문의	4칸만 넣으면 `user_id` 비어 있음 · `status` `RECEIVED` · `created_at` 채워짐 · 답변 칸 비어 있음
회원 문의	회원을 만들고 그 번호를 넣으면 저장됨
없는 회원 번호 (999999999)	DB 가 거절 (`IntegrityError`)
2,000자 글	그대로 저장되고 다시 읽힘

`tests/conftest.py` 정리 (모든 모델 시험이 같이 씀)

추가한 것	하는 일
`db`	실제 MySQL 세션. 연결할 수 없으면 시험을 건너뛰고, 끝나면 되돌림(rollback)
`assert_matches_db(모델)`	모델의 칸 이름 · 순서 · `NULL` 허용 · FK 를 실제 테이블과 비교

08 의 `tests/test_user_model.py` 에 있던 같은 코드를 여기로 옮겼고, 08 시험도 이것을 쓰도록 바꿨다. 남은 모델(10 ~)도 이것을 쓴다.

전체 시험 57개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 10 — `Project` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`Project` 클래스)
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	09 `Inquiry` 모델 · 11 `ProjectMember` 모델

1. 하는 일

MySQL 의 `projects` 테이블(시퀀서 프로젝트 한 개)을 파이썬 클래스 `Project` 로 옮겨 적는다. 08 · 09 와 같은 방식이고, 이번에 새로 나온 타입은 셋이다.

새로 나온 타입	칸	파이썬에서 읽으면
`DECIMAL(6,2)` (정확한 소수)	`bpm`	`Decimal("120.00")`
`TINYINT UNSIGNED` (0 ~ 255)	`time_sig_num` · `time_sig_den` (박자 4/4)	`4`
`JSON`	`master_settings` (마스터 채널 설정)	사전 `{"gain_db": -1.5}`

같은 파일의 `ProjectMember` 는 11, 관계(`project.members` · `project.tracks`)는 `ProjectMember` · `Track` 모델을 채운 뒤에 붙인다.

2. 언제 쓰나

쓰는 곳	화면	칸
P1-13 프로젝트 목록 · 만들기 · 이름 바꾸기 · 삭제	myProject.html	`owner_id` · `title` · `description` · `updated_at` · `deleted_at`
P1-16 대시보드 최근 프로젝트	myPage.html	`title` · `updated_at` · `thumbnail_path`
Phase 2 시퀀서	studioNx.html 위쪽 막대 (BPM · 박자 · 키)	`bpm` · `time_sig_*` · `music_key` · `scale_mode` · `sample_rate` · `master_settings` · `revision`

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from typing import Any

from sqlalchemy import JSON, BigInteger, FetchedValue, ForeignKey, Integer, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger` · `ForeignKey` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, 나머지를 이번에 추가했다.
`ProjectMember`(11)도 같은 import 를 쓴다.

import	가져오는 것	쓰는 곳
`Decimal` (파이썬 기본)	정확한 소수	`bpm`
`Any` (파이썬 기본)	"아무 타입" 표시	`dict[str, Any]` — JSON 안의 값
`Numeric(6, 2)`	SQL 의 `DECIMAL(6,2)`	`bpm`
`TINYINT` (MySQL 전용)	1바이트 정수	박자
`Integer`	SQL 의 `INT`	`sample_rate` · `revision`
`JSON`	SQL 의 `JSON`	`master_settings`
나머지	08 의 표 와 같음	

함수 (클래스 · 칸 선언)

```
class Project(Base):
    __tablename__ = "projects"

    project_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    owner_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    title: Mapped[str] = mapped_column(String(100))
    description: Mapped[str | None] = mapped_column(String(500))
    bpm: Mapped[Decimal] = mapped_column(Numeric(6, 2), server_default=text("120.00"))
    time_sig_num: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    time_sig_den: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    music_key: Mapped[str | None] = mapped_column(String(10))
    scale_mode: Mapped[str | None] = mapped_column(String(20))
    sample_rate: Mapped[int] = mapped_column(Integer, server_default=text("48000"))
    master_settings: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    revision: Mapped[int] = mapped_column(Integer, server_default=text("0"))
    thumbnail_path: Mapped[str | None] = mapped_column(String(500))
    # FK → project_templates (Phase 2). 그 모델이 생기면 ForeignKey 를 붙인다.
    template_id: Mapped[int | None] = mapped_column(BigInteger)
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL 과 비교 (앞에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`bpm DECIMAL(6,2) NOT NULL DEFAULT 120.00`	`Mapped[Decimal]` · `Numeric(6, 2)` · `server_default=text("120.00")`
`time_sig_num TINYINT UNSIGNED NOT NULL DEFAULT 4`	`Mapped[int]` · `TINYINT(unsigned=True)`
`master_settings JSON NULL`	`Mapped[dict[str, Any] | None]` · `JSON`
`template_id BIGINT UNSIGNED NULL` + (Phase 2 에서 `ALTER` 로 FK 추가)	`Mapped[int | None]` · **FK 는 적지 않음** — 아래 §5

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안의 MySQL 전용 타입

식	뜻
`Numeric(6, 2)`	전체 6자리, 소수점 아래 2자리 → 최대 `9999.99`
`Decimal("98.50")`	소수를 정확하게 저장. `float` 의 `0.1 + 0.2 = 0.30000000000000004` 같은 오차가 없음
`TINYINT(unsigned=True)`	0 ~ 255 정수
`dict[str, Any]`	열쇠는 글자, 값은 아무 타입인 사전. JSON 객체와 같은 모양
`mapped_column(JSON)`	저장할 때 사전 → JSON 글자, 읽을 때 JSON 글자 → 사전으로 자동으로 바꿈

4. 흐름 (새 프로젝트 하나를 넣을 때)

1. `Project(owner_id=7, title="My Song")` — 2칸만 넣는다.
2. `db.add` → `db.flush()` 로 `INSERT`.
3. DB 가 `project_id` · `bpm=120.00` · 박자 `4/4` · `sample_rate=48000` · `revision=0` · `created_at` 을 채운다.
4. P1-13 에서는 같은 저장 안에서 OWNER 멤버(11)와 MASTER 트랙(12)도 함께 넣는다 (FR-122).

5. 왜 이렇게

선택	이유
`bpm` 은 `Decimal`	화면에 `98.50` 을 넣으면 `98.50` 으로 돌아와야 한다. 입력 스키마(`schemas/project.py`)도 `Decimal` 이라 그대로 넘긴다
`template_id` 에 FK 를 적지 않음	가리키는 `project_templates` 모델이 아직 없다(Phase 2). 없는 테이블을 가리키면 SQLAlchemy 가 저장할 때 오류를 낸다. 실제 FK 는 DB 에 있으므로 잘못된 번호는 DB 가 막는다
`revision` 은 그냥 칸으로	저장할 때마다 +1 해서 동시 저장 충돌을 찾는 데 쓴다. 어떻게 올릴지는 저장 기능(Phase 2)에서 정한다
관계는 다음에	`project.members` · `project.tracks` 는 `ProjectMember`(11) · `Track`(12) 칸이 채워져야 붙일 수 있다
색인(`idx_projects_owner` · `idx_projects_updated`)은 적지 않음	08 · 09 와 같은 이유 — DB 의 검색용

6. 시험

`tests/test_project_model.py` (새로 만듦 · 4개) — 실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 17개 이름 · 순서 · `NULL` 허용 · FK(`owner_id` → `users`)
2칸만 넣기	`bpm` `120.00` · 박자 `4/4` · `48000` · `revision 0` · `master_settings` 비어 있음 · `created_at` 채워짐

시험	확인한 것
값을 넣고 DB 에서 다시 읽기	`bpm 98.50` · 박자 `3` · JSON `{"gain_db": -1.5, "limiter": {"on": true}}` 가 그대로
없는 회원 번호를 `owner_id` 로	DB 가 거절 (`IntegrityError`)

`tests/conftest.py` 의 `assert_matches_db` 보완

바꾼 것	이유
FK 의 `ON DELETE` 도 비교	11 `ProjectMember` 의 `ON DELETE CASCADE` 가 DB 와 같은지 보려고
아직 모델이 없는 테이블을 가리키는 FK 는 비교에서 뺌	`template_id → project_templates` (Phase 2)

전체 시험 61개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 11 — `ProjectMember` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`ProjectMember` 클래스)
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.1 · `studionx/db/01_phase1_account.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	10 `Project` 모델 · 12 `Track` 모델

1. 하는 일

MySQL 의 `project_members` 테이블(어느 회원이 어느 프로젝트에 어떤 권한으로 참여하는지)을 파이썬 클래스 `ProjectMember` 로 옮겨 적는다. 기본 키와 FK 2개는 뼈대에 이미 있었고, 이번에 나머지 3칸과 권한 목록 `MemberRole` 을 채웠다.

project_id	user_id	role	뜻
12	7	`OWNER`	7번 회원이 12번 프로젝트를 만들었다
12	9	`EDITOR`	9번 회원은 12번 프로젝트를 고칠 수 있다 (Phase 6 협업)

`role`	뜻
`OWNER`	만든 사람. 삭제 · 초대 가능
`EDITOR` (DB 기본값)	편집 가능
`VIEWER`	보기만

2. 언제 쓰나

쓰는 곳	하는 일
P1-13 프로젝트 만들기 (myProject.html **New project**)	프로젝트와 함께 `OWNER` 행 하나를 넣음 (FR-122)
P1-13 · P1-16 목록	"내가 멤버인 프로젝트" 를 찾을 때 이 테이블을 거침
Phase 6 협업	초대받은 회원을 `EDITOR` · `VIEWER` 로 추가, 톡방에서 읽은 위치 (`last_read_message_id`) 기록

Phase 1 에서는 **프로젝트마다 OWNER 한 행만** 쓴다.

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from enum import StrEnum
from typing import Any
```

```
from sqlalchemy import (
    JSON,
    BigInteger,
    Enum,
    FetchedValue,
    ForeignKey,
    Integer,
    Numeric,
    String,
    text,
)
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`project.py` 맨 위의 import 이다 (`Project` 와 함께 씀). 이번에 **`StrEnum`**·**`Enum`** 을 추가했다. 줄이 길어져 `ruff` 가 괄호로 묶어 한 줄에 하나씩 적었다. `ProjectMember` 는 이 중 `StrEnum` · `BigInteger` · `Enum` · `ForeignKey` · `text` · `DATETIME` · `Mapped` · `mapped_column` · `Base` 를 쓴다.

함수 (클래스 · 칸 선언)

```
class MemberRole(StrEnum):
    OWNER = "OWNER"
    EDITOR = "EDITOR"
    VIEWER = "VIEWER"


class ProjectMember(Base):
    __tablename__ = "project_members"

    # 복합 기본 키 (project_id, user_id)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"), primary_key=True)

    role: Mapped[MemberRole] = mapped_column(Enum(MemberRole), server_default=MemberRole.EDITOR)
    last_read_message_id: Mapped[int | None] = mapped_column(BigInteger)  # Phase 6 톡방
    joined_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

SQL 과 비교

SQL (`01_phase1_account.sql`)	파이썬
`PRIMARY KEY (project_id, user_id)`	두 칸 모두 `primary_key=True`
`FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCADE`	`ForeignKey("projects.project_id", ondelete="CASCADE")`
`role ENUM('OWNER','EDITOR','VIEWER') NOT NULL DEFAULT 'EDITOR'`	`Enum(MemberRole)` · `server_default=MemberRole.EDITOR`
`last_read_message_id BIGINT UNSIGNED NULL`	`Mapped[int | None]` (FK 없음 — DB 에도 없음)

SQL (01_phase1_account.sql)	파이썬
`joined_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`	`server_default=text(…)`

문법 풀이

import 문법	뜻
`from 모듈 import (A, B, …)`	가져올 이름이 많을 때 괄호로 묶어 여러 줄에 적음. 한 줄로 적은 것과 같음

식	뜻
복합 기본 키	두 칸을 **합쳐서** 한 행을 가리킴. `(12, 7)` 은 한 번만 있을 수 있고, `(12, 9)` · `(13, 7)` 은 따로 있을 수 있음
`db.get(ProjectMember, (12, 7))`	복합 기본 키로 찾을 때는 값을 튜플로 넘김
`ondelete="CASCADE"`	가리키는 프로젝트 행이 지워지면 **DB 가** 이 행도 함께 지움

4. 흐름 (프로젝트를 만들 때 · P1-13)

1. `Project(owner_id=7, title=…)` 를 넣고 `flush` → `project_id=12` 를 받는다.
2. `ProjectMember(project_id=12, user_id=7, role=MemberRole.OWNER)` 를 넣는다.
3. DB 가 `joined_at` 을 채운다.
4. 한 번에 `commit` 한다. 중간에 실패하면 둘 다 들어가지 않는다 (FR-122).

5. 왜 이렇게

선택	이유
기본 키를 `(project_id, user_id)` 로	같은 회원이 같은 프로젝트에 두 번 들어갈 수 없다. 따로 번호 칸이 필요 없다
`project_id` 에 `ON DELETE CASCADE`	프로젝트 행이 정말 지워지면 멤버 행만 남을 이유가 없다. 보통 삭제는 `deleted_at` 기록(소프트 삭제)이라 이 경우는 드물다 — 관리자가 DB 에서 직접 지울 때 등
`user_id` 에는 `CASCADE` 없음	회원은 지우지 않고 `status=DELETED` 로 바꾼다 (08). 회원 행을 실수로 지우려 하면 DB 가 막는다
`role` 기본값이 `EDITOR`	Phase 6 에서 초대받은 사람이 보통 편집자이기 때문. 만든 사람은 코드에서 `OWNER` 를 꼭 적는다
`last_read_message_id` 에 FK 없음	DB 에도 없다. 가리킬 `chat_messages` 는 Phase 6 이고, 메시지가 지워져도 읽은 위치 숫자는 남아도 된다

6. 시험

`tests/test_project_member_model.py` (새로 만듦 · 5개) — 실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지	칸 5개 · `NULL` 허용 · FK 2개 (`project_id` 는 `ON DELETE CASCADE` 까지)
기본 키	`["project_id", "user_id"]`

시험	확인한 것
OWNER · 기본값	`role=OWNER` 를 적으면 `OWNER`, 안 적으면 DB 가 `EDITOR` · `joined_at` 채움 · `last_read_message_id` 비어 있음
같은 회원을 두 번	DB 가 거절 (`IntegrityError`)
프로젝트 행을 지우면	멤버 수 1 → 0 (DB 의 `CASCADE`)

전체 시험 66개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 12 — `Track` 모델 컬럼 (Phase 1 칸만)

항목	내용
파일	`studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통 — ORM 모델 컬럼 · `docs/DB.md` §3.2 · `studionx/db/02_phase2_sequencer.sql`
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	11 `ProjectMember` 모델 · 13 관계

1. 하는 일

MySQL 의 `tracks` 테이블(시퀀서의 트랙 한 줄)을 파이썬 클래스 `Track` 으로 옮겨 적는다. 테이블에는 칸이 19개 있지만, **Phase 1 에 필요한 3칸만** 채웠다. 나머지 14칸(색 · 볼륨 · 음소거 등)은 시퀀서를 만드는 Phase 2 에서 채운다.

지금 모델에 있는 칸	뜻
`track_id` · `project_id` (뼈대에 있었음)	트랙 번호 · 속한 프로젝트
`name`	트랙 이름. 예: `Master`
`track_type`	종류. `AUDIO` · `MIDI` · `MASTER` · `GROUP` · `RETURN`
`order_index`	화면에서의 순서 (0부터)

2. 언제 쓰나

Phase 1 에서는 **프로젝트를 만들 때 MASTER 트랙 1개를 넣는 데만** 쓴다 (P1-13, FR-122). studioNx.html 을 처음 열면 맨 아래에 보이는 Master 트랙이 이 행이다.

```
# P1-13 project_service.create_project() 안 (예정)
db.add(Track(project_id=project.project_id, name="Master", track_type=TrackType.MASTER, order_index=0))
```

프로젝트를 만들 때 함께 넣는 것	모델
프로젝트 행	10 `Project`
OWNER 멤버 행	11 `ProjectMember`
MASTER 트랙 행	**12 `Track`**

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, ForeignKey, Integer, String, text
from sqlalchemy.orm import Mapped, mapped_column
```

```
from app.db.session import Base
```

`BigInteger` · `ForeignKey` · `Mapped` · `mapped_column` · `Base` 는 원래 있었고, **`StrEnum`** · **`Enum`** · **`Integer`** · **`String`** · **`text`** 를 이번에 추가했다. 모두 이 클래스에서 쓴다. 시각 칸(`created_at` 등)을 아직 넣지 않아서 `DATETIME` 은 없다.

함수 (클래스 · 칸 선언)

```
class TrackType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"
    MASTER = "MASTER"
    GROUP = "GROUP"
    RETURN = "RETURN"


class Track(Base):
    __tablename__ = "tracks"

    track_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )

    # Phase 1 칸 (MASTER 트랙 생성용). 나머지 칸은 DB 기본값 · NULL 로 채워진다
    name: Mapped[str] = mapped_column(String(100))
    track_type: Mapped[TrackType] = mapped_column(Enum(TrackType))
    order_index: Mapped[int] = mapped_column(Integer, server_default=text("0"))

    # TODO(Phase 2): color, volume_db, pan, is_muted, is_solo, … (뼈대의 주석 그대로)
```

SQL 과 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`name VARCHAR(100) NOT NULL`	`Mapped[str]` · `String(100)`
`track_type ENUM('AUDIO',…,'RETURN') NOT NULL` (**기본값 없음**)	`Enum(TrackType)` · `server_default` 없음
`order_index INT UNSIGNED NOT NULL DEFAULT 0`	`Integer` · `server_default=text("0")`
`color CHAR(7) NOT NULL DEFAULT '#7C5CFF'` 외 13칸	적지 않음 (Phase 2) — 넣을 때 DB 가 기본값 · `NULL` 로 채움

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
모델에 없는 칸	SQLAlchemy 는 모델에 적힌 칸만 `INSERT` · `SELECT` 한다. 나머지 칸은 DB 가 기본값으로 채우고, 파이썬에서는 `track.color` 처럼 읽을 수 없다

식	뜻
`track_type` 에 `server_default` 없음	반드시 넣어야 하는 칸. 빠뜨리면 DB 가 거절

4. 흐름 (MASTER 트랙을 넣을 때)

1. `Track(project_id=12, name="Master", track_type=TrackType.MASTER)` 를 만든다.
2. `db.add` → `db.flush()` 로 `INSERT INTO tracks (project_id, name, track_type) …`.
3. DB 가 `track_id` · `order_index=0` 과, 모델에 없는 칸(`color='#7C5CFF'` · `volume_db=0.00` · `is_muted=0` · `routing_mode='CHAIN'` …)을 채운다.

5. 왜 이렇게

선택	이유
Phase 1 칸만 채움	뼈대 TODO 가 Phase 1 칸만 정해 두었다. 나머지 칸은 시퀀서(Phase 2)에서 의미가 정해지고, 지금 적으면 쓰지 않는 코드가 된다. 모두 DB 기본값이나 `NULL` 이 있어 넣을 때 문제가 없다
`track_type` 에 기본값을 두지 않음	DB 에도 없다. 트랙 종류는 만드는 쪽이 꼭 정해야 한다
FK 는 `project_id` 하나만	DB 에는 FK 가 5개 있지만, 나머지 4개(`parent_track_id` · `frozen_sample_id` · `assignee_id` · `locked_by`)는 Phase 2 칸에 붙어 있다. 그 칸을 채울 때 함께 적는다
색인 (`idx_tracks_project_order`)은 적지 않음	08 ~ 11 과 같은 이유

6. 시험

`tests/test_track_model.py` (새로 만듦 · 4개) — 실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB 와 같은지 (모델의 5칸만)	이름 · 순서 · `NULL` 허용 · `project_id` FK 의 `ON DELETE CASCADE`
MASTER 트랙	`order_index` 0 (DB 기본값). 모델에 없는 칸도 SQL 로 읽어 보니 `#7C5CFF` · `0.00` · `0` · `CHAIN` 으로 채워져 있음
`track_type` 을 빠뜨리면	DB 가 거절 (`IntegrityError`)
프로젝트 행을 지우면	트랙 수 1 → 0 (DB 의 `CASCADE`)

`tests/conftest.py` 의 `assert_matches_db` 보완: `partial=True` 를 주면 모델에 있는 칸만 비교하고, 그 칸에 붙은 FK 만 비교한다. 다른 모델은 그대로 전체를 비교한다.

전체 시험 70개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 13 — 모델 관계 (`project.members` · `project.tracks`)

항목	내용
파일	`studionx/backend/app/models/project.py` · `studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통 — ORM 모델 관계 (P1-03 마지막)
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	12 `Track` 모델 · 14 `check_password_rules()` (P1-04)

1. 하는 일

10 ~ 12 의 FK(`project_id`)를 **파이썬 속성**으로도 오갈 수 있게 한다. FK 는 DB 의 숫자 연결이고, 관계(`relationship`)는 그 연결을 파이썬 객체로 따라가게 해 준다.

관계가 없을 때	관계가 있을 때
`db.scalars(select(Track).where(Track.project_id == 12).order_by(Track.order_index))`	`project.tracks`
`db.get(Project, member.project_id)`	`member.project`
프로젝트를 `flush` 해서 번호를 받은 뒤 `ProjectMember(project_id=12, …)`	`project.members.append(ProjectMember(…))` — 번호는 저장할 때 자동으로 채워짐

속성	방향	값
`project.members`	프로젝트 → 멤버	`[ProjectMember, …]`
`project.tracks`	프로젝트 → 트랙	`[Track, …]` — `order_index` 순서
`member.project` · `track.project`	반대 방향	`Project`

2. 언제 쓰나

쓰는 곳	쓰는 방법
P1-13 프로젝트 만들기 (FR-122)	프로젝트에 OWNER 멤버 · MASTER 트랙을 붙여 `db.add(project)` 한 번 → 셋이 함께 저장
Phase 2 시퀀서	studioNx.html 을 열 때 `project.tracks` 로 트랙 목록을 화면 순서대로 읽음
Phase 6 협업	`project.members` 로 멤버 목록 · 권한 확인

```
# P1-13 create_project() 를 관계로 쓰면
project = Project(owner_id=user.user_id, title="My Song")
project.members.append(ProjectMember(user_id=user.user_id, role=MemberRole.OWNER))
project.tracks.append(Track(name="Master", track_type=TrackType.MASTER, order_index=0))
db.add(project)
db.commit()   # 하나라도 실패하면 셋 다 저장되지 않음
```

3. 코드

import (`project.py` 맨 위 — 바뀐 줄만)

```
from typing import TYPE_CHECKING, Any
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음 – 서로 불러오는 순환을 피함)
    from app.models.track import Track
```

import (`track.py` 맨 위 — 바뀐 줄만)

```
from typing import TYPE_CHECKING
from sqlalchemy.orm import Mapped, mapped_column, relationship

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음)
    from app.models.project import Project
```

두 파일 모두 **`relationship`** · **`TYPE_CHECKING`** 을 추가했다. `project.py` 는 `Track` 을, `track.py` 는 `Project` 를 타입 표시에만 쓴다.

함수 (`project.py` 의 `Project` 끝 · `ProjectMember` 끝)

```
class Project(Base):
    ...
    # 관계 – 행을 지우는 일은 DB 의 ON DELETE CASCADE 에 맡김 (passive_deletes)
    members: Mapped[list["ProjectMember"]] = relationship(
        back_populates="project", cascade="all, delete-orphan", passive_deletes=True
    )
    tracks: Mapped[list["Track"]] = relationship(
        back_populates="project",
        order_by="Track.order_index",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )


class ProjectMember(Base):
    ...
    project: Mapped[Project] = relationship(back_populates="members")
```

(`track.py` 의 `Track` 끝)

```
    project: Mapped["Project"] = relationship(back_populates="tracks")
```

문법 풀이

import 문법	뜻
`from typing import TYPE_CHECKING`	편집기 · 타입 검사기가 읽을 때만 `True`, 실제로 실행할 때는 `False` 인 값

import 문법	뜻
`if TYPE_CHECKING: from … import Track`	타입 표시에만 쓰는 import. `project.py` 와 `track.py` 가 서로를 실행 중에 불러오면 "아직 다 읽지 않은 파일" 오류(순환 import)가 날 수 있어 피함

식	뜻
`relationship()`	FK 를 따라 연결된 객체를 속성으로 줌. 어느 FK 를 쓸지는 SQLAlchemy 가 칸을 보고 찾음
`Mapped[list["ProjectMember"]]`	여러 개 (목록). `"…"` 글자로 적으면 아직 정의되지 않은 클래스도 가리킬 수 있음
`Mapped[Project]`	하나
`back_populates="project"`	반대쪽 속성 이름. 한쪽을 바꾸면 다른 쪽도 맞춰짐 (`member.project` ↔ `project.members`)
`order_by="Track.order_index"`	`project.tracks` 를 읽을 때 이 순서로 정렬
`cascade="all, delete-orphan"`	프로젝트를 `add` · `delete` 하면 자식도 함께. 목록에서 뺀 자식(고아)은 행도 지움
`passive_deletes=True`	프로젝트를 지울 때 자식 행을 파이썬이 하나씩 지우지 않고 DB 의 `ON DELETE CASCADE` 에 맡김

4. 흐름 (`db.add(project)` → `flush`)

1. `project.members` · `project.tracks` 에 붙은 객체도 함께 저장 대상이 된다 (`cascade`).
2. `INSERT INTO projects` → `project_id=12` 를 받는다.
3. 자식의 `project_id` 칸에 12 를 자동으로 넣고 `INSERT INTO project_members` · `INSERT INTO tracks`.
4. 하나라도 실패하면 `rollback` 으로 셋 다 없던 일이 된다.

5. 왜 이렇게

선택	이유
`members` 는 뼈대 TODO 그대로 (`cascade="all, delete-orphan"`)	멤버는 프로젝트 없이 존재할 수 없다
`tracks` 에도 `cascade` 를 추가 (뼈대 TODO 에는 `order_by` 만)	없으면 트랙을 읽은 상태로 프로젝트를 지울 때, SQLAlchemy 가 트랙의 `project_id` 를 비우려다 `NOT NULL` 오류를 낸다. 트랙도 프로젝트 없이 존재할 수 없다
`passive_deletes=True`	DB 에 이미 `ON DELETE CASCADE` 가 있다(11 · 12). 파이썬이 자식을 모두 읽어 하나씩 지우면 느리고 같은 일을 두 번 한다
보통 삭제는 소프트 삭제	P1-13 의 삭제는 `deleted_at` 기록이라 위 삭제 동작은 쓰이지 않는다. 관리자가 정말 지우는 경우를 위한 것
`User.projects` 는 아직 안 넣음	뼈대에 "필요해지면" 으로 되어 있다. 목록은 `ProjectMember` 로 JOIN 해서 찾는다 (P1-13 TODO)
`TYPE_CHECKING` 으로 import	두 파일이 서로를 가리키므로, 실행 중 import 순환을 피하면서 편집기 자동 완성은 살린다

6. 시험 (`tests/test_model_relationships.py`, 새로 만듦 · 6개)

실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
프로젝트만 `add`	멤버 · 트랙도 저장되고, 자식의 `project_id` 가 자동으로 채워짐
반대 방향	`member.project is project` · `track.project is project`
트랙 순서	`order_index` 2 · 1 순서로 붙여도 다시 읽으면 `Master` · `Bass` · `Drums`
멤버를 목록에서 빼면	행이 지워짐 (`delete-orphan`)
자식을 읽은 상태로 프로젝트를 지우면	오류 없이 멤버 · 트랙 0개
트랙 하나가 실패하면 (`track_type` 빠짐)	`IntegrityError` → `rollback` 후 프로젝트도 없음 (FR-122)

전체 시험 76개 통과, `ruff` 검사 통과. 앱 시작 시 관계 설정(`configure_mappers()`)도 오류 없음.